

Catalyst Intent
Security Review

Cantina Managed review by:

Noah Marconi, Lead Security Researcher

Phaze, Lead Security Researcher
Cccz, Security Researcher

April 2, 2025

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**

3 Findings **4**
- 3.1 High Risk . . . 4
  - 3.1.1 Missing timestamp validation allows frontrunning of disputes . . . 4
- 3.2 Medium Risk . . . 5
  - 3.2.1 Malicious input and output tokens may compromise the solver . . . 5
- 3.3 Low Risk . . . 5
  - 3.3.1 Finalized orders can still be purchased . . . 5
  - 3.3.2 Incorrect length checks prevent `PolymerOracle` initialization . . . 6
  - 3.3.3 Invalid topic processing in PolymerOracle . . . 7
  - 3.3.4 Bitcoin Oracle proof can be reused for multiple orders . . . 7
  - 3.3.5 Limited flexibility and precision in BTC-to-collateral price multiplier . . . 8
  - 3.3.6 BtcPrism security considerations with confirmation requirements . . . 9
  - 3.3.7 BTC timestamp manipulation can block valid order verification . . . 10
  - 3.3.8 `_getProofPeriod()` returns incorrect period . . . 10
  - 3.3.9 Zero-division error causes `CoinFillerWithFee` DoS . . . 10
  - 3.3.10 Duplicate outputs are only filled once in orders . . . 11
- 3.4 Gas Optimization . . . 11
  - 3.4.1 Code optimization suggestions . . . 11
- 3.5 Informational . . . 12
  - 3.5.1 OP Stack interoperability risks for fills and resource unlocks . . . 12
  - 3.5.2 Add `remoteChainId` non-zero check in `PolymerOracle._processMessage()` . . . 13
  - 3.5.3 `BitcoinOracle` should limit the maximum number of confirmations . . . 13
  - 3.5.4 Code quality improvements . . . 13

DRAFT

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	*Must* fix as soon as possible (if already deployed).
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

DRAFT

2 Security Review Summary

LI.FI is a cross-chain bridge aggregation protocol that supports any-2-any swaps by aggregating bridges and connecting them to DEX aggregators.

From Mar 11th to Mar 27th the Cantina team conducted a review of catalyst-intent on commit hash fcdbdc6a. The team identified a total of **17** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	1	0	1
Low Risk	10	6	4
Gas Optimizations	1	1	0
Informational	4	2	2
Total	**17**	**10**	**7**

DRAFT

3 Findings

3.1 High Risk

3.1.1 Missing timestamp validation allows frontrunning of disputes

Severity: High Risk

Context: BitcoinOracle.sol#L521-L544

Summary: The `BitcoinOracle` contract lacks a timestamp validation check that allows malicious solvers to profit by front-running disputes. Solvers can claim intents without fulfilling them, wait for disputes, then quickly fulfill before disputes finalize, receiving both their collateral and part of the disputer's collateral.

Description: The `BitcoinOracle` contract's dispute mechanism does not properly verify that the Bitcoin transaction used to fulfill an intent was included before the dispute was filed. This allows a malicious solver to claim an intent without actually fulfilling it, and then rush to fulfill it after someone disputes their claim, potentially causing the disputer to lose their collateral. The key issue is that in the `_resolveClaimed()` function, when determining if a proof is valid, there is no check to verify that the transaction timestamp is before the dispute timestamp:

```
if (sponsor != address(0) && fillTimestamp >= claimTimestamp) {
    bool disputed = disputer != address(0);
    // ... pay out collateral to solver
}
```

This condition should also verify that the transaction occurred before the dispute was registered by checking `fillTimestamp <= disputeTimestamp`. This vulnerability enables the following attack scenario:

1. A malicious solver claims an intent without actually fulfilling it.
2. A legitimate solver notices this and disputes the claim, depositing 2x collateral.
3. The malicious solver quickly creates a fulfilling transaction after seeing the dispute.
4. If the malicious solver gets their transaction included in the blockchain and verified before the disputer can finalize the dispute, they will receive their collateral plus most of the disputer's collateral.

DRAFT

The documentation in the code comments suggests this check was intended but not implemented:

```
/**
 * @notice For a claim to payout to the sponsor, it is required that the input was included before this
 ↪  timestamp.
 * For disputers, note that it is possible to set the inclusion timestamp to 1 block prior.
 * @dev Is the maximum of (block.timestamp and claimTimestamp + MIN_TIME_FOR_INCLUSION)
 */
uint32 disputeTimestamp;
```

Impact Explanation: The impact is high. Malicious solvers can profit at the expense of honest disputers, who lose their 2x collateral when challenging fraudulent claims. This creates an incentive structure where disputing fraudulent claims becomes financially risky, undermining the security of the optimistic verification system.

Likelihood Explanation: The likelihood is medium. The attack requires specific timing but offers strong economic incentives (potentially 2x collateral gain). Feasibility varies with the required confirmation count—lower confirmation requirements make the attack more viable, while higher requirements make it harder but still possible.

Recommendation: Add validation in the `_resolveClaimed()` function to ensure that the transaction timestamp is before the dispute timestamp, and only apply the dispute logic when a disputer exists:

```
function _resolveClaimed(uint32 fillTimestamp, bytes32 orderId, OutputDescription calldata output) internal
↪  returns (bytes32 solver) {
    bytes32 outputId = _outputIdentifier(output);
    ClaimedOrder storage claimedOrder = _claimedOrder[orderId][outputId];
    solver = claimedOrder.solver;
    if (solver == bytes32(0)) revert NotClaimed();

    // Check if there are outstanding collateral associated with the
    // registered claim.
    address sponsor = claimedOrder.sponsor;
    uint32 claimTimestamp = claimedOrder.claimTimestamp;
    uint96 multiplier = claimedOrder.multiplier;
    address disputer = claimedOrder.disputer;
+   uint32 disputeTimestamp = claimedOrder.disputeTimestamp;

-   if (sponsor != address(0) && fillTimestamp >= claimTimestamp) {
+   if (sponsor != address(0) && fillTimestamp >= claimTimestamp && (disputer == address(0) || fillTimestamp <=
↪  disputeTimestamp)) {
        bool disputed = disputer != address(0);
        // ... rest of function
    }
}
```

This change ensures that the solver can only claim rewards if either:

1. No dispute has been registered, or...
2. The Bitcoin transaction was executed before the dispute was registered.

Further, consider adding a small safety buffer to the dispute timestamp check and consider removing the ability to use the previous block's timestamp, as this feature potentially increases the likelihood of an exploit by giving attackers more flexibility in meeting timestamp requirements.

LiFi: Fixed in PR 91.

Cantina Managed: Fix verified.

3.2 Medium Risk

3.2.1 Malicious input and output tokens may compromise the solver

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: When the order contains malicious input or output tokens, the solver may be compromised due to the inability to finalize the order, or partially fill the order. For input tokens, when the solver fills the order, that solver can call the `finalise*()` function to claim the assets in the resource lock, where `COMPACT.claimAndWithdraw()` sends all the input assets to the solver. The problem here is that if any of the assets sent fails, the entire transaction will revert, resulting in the solver not being able to claim the input assets. Consider the following scenario:

1. Alice creates a resource lock with input tokens of 10,000 USDC and 1 XYZ token, and output tokens of 5 ETH. where the XYZ token is a malicious token controlled by Alice.
2. Bob, the solver, fills the order.
3. Alice upgrades the implementation of the XYZ token so that the transfer will fail.
4. Bob finalizes the order, but is unable to claim the 10,000 USDC because the XYZ token failed to be sent.
5. After the resource lock expires, Alice upgrades the XYZ token and withdraws the 10,000 USDC by forcing withdrawal.

For output tokens, an order may have multiple outputs, and the solver that fills the first output of the order can finalize the order on the source chain. The solver may call `fillSkip()` to competitively batch fills `output[0]` for multiple orders to grab orders, and then fill the subsequent outputs. Once the subsequent output cannot be filled, the proof cannot be written and the order cannot be finalized. Consider the following scenario:

DRAFT

1. Most tokens (such as wstETH or tokens inherited from OZ ERC20) no longer allow 0 address transfers. So consider the first output of the order is to transfer 1 BTC to `address(alice)`, and the second output is to transfer 0 wstETH to `address(0)` (Or transferring USDC to a USDC blacklisted address).
2. Filling the second output of the order will always fail, resulting in the proof being unable to be written and the order unable to be finalized.

Proof of Concept:

```
// ...
vm.mockCall(
    address(token),
    abi.encodeWithSelector(ERC20.transfer.selector),
    abi.encode(false)
);
vm.prank(solver);
compactSettler.finaliseSelf(order, signature, timestamps, solverIdentifier); // revert TransferFailed
vm.snapshotGasLastCall("finaliseSelf");
```

Recommendation: It is recommended to document the issue to inform the user and on the front-end to check the order and warn the user.

LiFi: Acknowledged.

Cantina Managed: Acknowledged.

3.3 Low Risk

3.3.1 Finalized orders can still be purchased

Severity: Low Risk

Context: BaseSettler.sol#L140-L152

Summary: The `CompactSettler` contract lacks a mechanism to prevent the purchase of orders that have already been finalized. This allows a malicious solver to front-run a purchaser's transaction by finalizing the order first, then allowing the purchase to go through, effectively stealing the purchaser's funds.

DRAFT

Description: In the current implementation, when an order is finalized using `finaliseSelf()` or related functions, the contract emits a `Finalised` event but does not record the state of the order as "*fulfilled*" in a way that can be checked during purchase.

This creates an opportunity where:

1. A solver who is aware of an incoming purchase transaction can front-run it by first finalizing the order for themselves.
2. When the purchaser's transaction is processed, the purchase succeeds despite the order having already been filled.
3. The purchaser pays for an order they cannot fulfill, resulting in direct loss of funds.

In effect, the `purchaseOrder()` function lacks a check to verify if the order has been already fulfilled, leading to this attack vector.

Impact Explanation: The impact of this vulnerability is high. It enables solvers to steal funds from users who attempt to purchase their orders. When exploited, purchasers lose the tokens they used to purchase the order, while receiving nothing in return since the order has already been fulfilled.

Likelihood Explanation: The likelihood is low. Exploiting this vulnerability requires a malicious solver to finalize the order before a victim is able to purchase it.

While this attack pattern requires specific timing, a sophisticated solver could delay providing purchase signatures until they are ready to finalize and submit proofs to the oracle and immediately finalize the order.

Proof of Concept: The test case demonstrates this vulnerability:

```
function test_poc_finalise_self_front_running_purchase_order() external {
    vm.prank(swapper);
    uint256 amount = 1e18 / 10;
    uint256 tokenId = theCompact.deposit(address(token), alwaysOKAllocator, amount);
```

```
address localOracle = address(alwaysYesOracle);

uint256[2][] memory inputs = new uint256[2][](1);
inputs[0] = [tokenId, amount];
OutputDescription[] memory outputs = new OutputDescription[](1);
outputs[0] = OutputDescription({
    remoteFiller: bytes32(uint256(uint160(address(coinFiller)))),
    remoteOracle: bytes32(uint256(uint160(localOracle))),
    chainId: block.chainid,
    token: bytes32(uint256(uint160(address(anotherToken)))),
    amount: amount,
    recipient: bytes32(uint256(uint160(swapper))),
    remoteCall: hex"",
    fulfillmentContext: hex""
});
CatalystCompactOrder memory order =
    CatalystCompactOrder({ user: address(swapper), nonce: 0, originChainId: block.chainid, fillDeadline:
    ↪  type(uint32).max, localOracle: alwaysYesOracle, inputs: inputs, outputs: outputs });

// Make Compact
bytes32 typeHash = TheCompactOrderType.BATCH_COMPACT_TYPE_HASH;
uint256[2][] memory idsAndAmounts = new uint256[2][](1);
idsAndAmounts[0] = [tokenId, amount];

bytes memory sponsorSig = getCompactBatchWitnessSignature(swapperPrivateKey, typeHash,
↪  address(compactSettler), swapper, 0, type(uint32).max, idsAndAmounts, this.orderHash(order));
bytes memory allocatorSig = hex"";
bytes memory signature = abi.encode(sponsorSig, allocatorSig);

bytes32 solverIdentifier = bytes32(uint256(uint160((solver))));
bytes32 orderId = compactSettler.orderIdentifier(order);

uint32[] memory timestamps = new uint32[](1);
timestamps[0] = uint32(block.timestamp);

// --- Purchase Order ---
address purchaser = address(0x123123123);
token.mint(purchaser, amount);
vm.prank(purchaser);
token.approve(address(compactSettler), amount);
vm.prank(purchaser);
anotherToken.approve(address(compactSettler), amount);

address newDestination = solver;
uint64 discount = 0;
uint32 timeToBuy = 1000;
uint256 expiryTimestamp = type(uint256).max;
bytes memory call = hex"";
bytes memory solverSignature = this.getOrderPurchaseSignature(solverPrivateKey, orderId,
↪  address(compactSettler), newDestination, call, discount, timeToBuy);

uint256 snapshot = vm.snapshot();

// --- Expected Flow ---
// Purchaser is able to purchase the order.
vm.prank(purchaser);
compactSettler.purchaseOrder(orderId, order, solverIdentifier, purchaser, expiryTimestamp, newDestination,
↪  call, discount, timeToBuy, solverSignature);

// Purchaser is able to finalise their purchased order.
vm.prank(purchaser);
compactSettler.finaliseSelf(order, signature, timestamps, solverIdentifier);

// Solver and purchaser get their tokens back.
assertEq(token.balanceOf(solver), amount);
assertEq(token.balanceOf(purchaser), amount);

vm.revertTo(snapshot);

// --- Unexpected Flow ---
// Solver finalises the order before the purchaser has a chance to.
vm.prank(solver);
compactSettler.finaliseSelf(order, signature, timestamps, solverIdentifier);

// Purchaser transaction is received after order is finalised.
```

DRAFT

```
    vm.prank(purchaser);
    compactSettler.purchaseOrder(orderId, order, solverIdentifier, purchaser, expiryTimestamp, newDestination,
    ↪  call, discount, timeToBuy, solverSignature);

    // Purchaser cannot finalise their purchased order.
    vm.prank(purchaser);
    vm.expectRevert();
    compactSettler.finaliseSelf(order, signature, timestamps, solverIdentifier);

    // Solver gets the tokens of the purchaser.
    assertEq(token.balanceOf(solver), 2 * amount);
    assertEq(token.balanceOf(purchaser), 0);
}
```

Recommendation: Implement a mapping to track fulfilled orders and add a check in the `purchaseOrder()` function to prevent purchasing already fulfilled orders:

```
+ // Mapping to track fulfilled orders
+ mapping(bytes32 => bool) public fulfilledOrders;

  function _finalise(CatalystCompactOrder calldata order, bytes calldata signatures, bytes32 orderId, bytes32
  ↪  solver, address destination) internal {
      bytes calldata sponsorSignature = BytesLib.toBytes(signatures, 0);
      bytes calldata allocatorSignature = BytesLib.toBytes(signatures, 1);
      // Payout inputs. (This also protects against re-entry calls.)
      _resolveLock(order, sponsorSignature, allocatorSignature, destination);

+     // Mark order as fulfilled
+     fulfilledOrders[orderId] = true;

      emit Finalised(orderId, solver, destination);
  }

  function purchaseOrder(
      bytes32 orderId,
      CatalystCompactOrder calldata order,
      bytes32 orderSolvedByIdentifier,
      address purchaser,
      uint256 expiryTimestamp,
      address newDestination,
      bytes calldata call,
      uint64 discount,
      uint32 timeToBuy,
      bytes calldata solverSignature
  ) external {
      // Sanity check that the user thinks they are buying the right order.
      bytes32 computedOrderId = _orderIdentifier(order);
      if (computedOrderId != orderId) revert OrderIdMismatch(orderId, computedOrderId);

+     // Check if the order has already been fulfilled
+     if (fulfilledOrders[orderId]) revert OrderAlreadyFulfilled();

      uint256[2][] calldata inputs = order.inputs;
      _purchaseOrder(orderId, inputs, orderSolvedByIdentifier, purchaser, expiryTimestamp, newDestination,
      ↪  call, discount, timeToBuy, solverSignature);
  }
```

DRAFT

This would prevent the purchase of already filled orders, protecting users from the possibility of a front-running attack.

LiFi: Acknowledged. I understand the attack vector and acknowledge it. The intended usage of purchase is very early in an order's life. Considering that most validation layers takes more than 10-30 minutes to finalize, that use case of purchase is not affected.

As a result, I would rather say that this is a purchaser issue (i.e. they need to make sure they purchase orders before the proof arrives) than a contract issue. However, I will take the recommendation into consideration.

Cantina Managed: Acknowledged.

3.3.2 Incorrect length checks prevent `PolymerOracle` initialization

Severity: Low Risk

Context: PolymerOracle.sol#L40-L54

Description: The `PolymerOracle` contract contains erroneous encoding length checks in the `setChainMap()` function that prevent the contract from being initialized properly. The flawed validation logic renders the contract unusable as it blocks the mapping between messaging protocol chain identifiers and blockchain IDs.

The problematic code is:

```
if (abi.encodePacked(messagingProtocolChainIdentifier).length == 0) revert ZeroValue();
// ...
if (abi.encodePacked(_blockChainIdToChainIdentifier[chainId]).length != 0) revert AlreadySet();
```

These checks are incorrect because:

1. `messagingProtocolChainIdentifier` is a `uint32` value which, when packed, always has a length of 4 bytes, regardless of its value (even if it's zero). The first condition will never trigger the `ZeroValue()` error because `abi.encodePacked(0)` produces `0x00000000` (4 bytes).
2. Similarly, the second check always reverts. When reading from a mapping with a non-existent key, Solidity returns the default value for that type (0 for `uint32`). When this zero value is encoded with `abi.encodePacked()`, it produces a 4-byte result, not an empty byte array. This means `abi.encodePacked(_blockChainIdToChainIdentifier[chainId]).length` is always 4, not 0, causing the function to always revert with `AlreadySet()`.

Recommendation: Replace the flawed encoding length checks with direct value comparisons:

```
  function setChainMap(uint32 messagingProtocolChainIdentifier, uint256 chainId) external onlyOwner {
      // Check that the inputs haven't been mistakenly called with 0 values.
-     if (abi.encodePacked(messagingProtocolChainIdentifier).length == 0) revert ZeroValue();
+     if (messagingProtocolChainIdentifier == 0) revert ZeroValue();
      if (chainId == 0) revert ZeroValue();

      // This call only allows setting either value once, then they are done for.
      // We need to check if they are currently unset.
      if (_chainIdentifierToBlockChainId[messagingProtocolChainIdentifier] != 0) revert AlreadySet();
-     if (abi.encodePacked(_blockChainIdToChainIdentifier[chainId]).length != 0) revert AlreadySet();
+     if (_blockChainIdToChainIdentifier[chainId] != 0) revert AlreadySet();

      _chainIdentifierToBlockChainId[messagingProtocolChainIdentifier] = chainId;
      _blockChainIdToChainIdentifier[chainId] = messagingProtocolChainIdentifier;

      emit MapMessagingProtocolIdentifierToChainId(messagingProtocolChainIdentifier, chainId);
  }
```

LiFi: Fixed in PR 83.

Cantina Managed: Fix verified.

3.3.3 Invalid topic processing in PolymerOracle

Severity: Low Risk

Context: PolymerOracle.sol#L86

Description: The `PolymerOracle` contract has an implementation issue with how it processes event topics from Polymer proofs. The current code attempts to extract the `orderId` directly from the first position of the topics byte array:

```
bytes32 orderId = bytes32(topics[0]);
```

This approach is incorrect for two reasons:

1. According to Polymer's documentation and API, the `topics` returned from `validateEvent()` is a concatenated byte array of all event topics, not an array of individual topics. Proper parsing requires either assembly or using ABI decoding with length checks to extract each 32-byte topic.
2. In the context of the intended event structure, `OutputFilled` does not have `orderId` as an indexed parameter:

```
event OutputFilled(bytes32 orderId, bytes32 solver, uint32 timestamp, OutputDescription output);
```

DRAFT

This means the `orderId` would not be in the topics at all but rather in the unindexed data.

Recommendation: Modify the topic processing to properly extract indexed topics using assembly, and decode the unindexed data correctly:

```
  function _processMessage(
      bytes calldata proof
  ) internal {
      (uint32 chainId, address emittingContract, bytes memory topics, bytes memory unindexedData) =
      ↪  CROSS_L2_PROVER.validateEvent(proof);

+     // Verify this is the correct event type (first 32 bytes of topics)
+     require(topics.length == 32, "Invalid topics length");
+     bytes32 eventSignature;
+     assembly {
+         // Skip first 32 bytes (length prefix of bytes array)
+         eventSignature := mload(add(topics, 32))
+     }
+
+     // Verify this is the OutputFilled event
+     require(eventSignature == OutputFilled.selector, "Invalid event signature");

-     // Store payload attestations;
-     bytes32 orderId = bytes32(topics[0]);
-     (bytes32 solver, uint32 timestamp, OutputDescription memory output) = abi.decode(unindexedData, (bytes32,
↪  uint32, OutputDescription));

+     // Decode the unindexed data containing all parameters
+     (bytes32 orderId, bytes32 solver, uint32 timestamp, OutputDescription memory output) = abi.decode(
+         unindexedData,
+         (bytes32, bytes32, uint32, OutputDescription)
+     );

      bytes32 payloadHash = _proofPayloadHash(orderId, solver, timestamp, output);

      // Convert the Polymer ChainID into the canonical chainId.
      uint256 remoteChainId = _chainIdentifierToBlockChainId[chainId];
      bytes32 senderIdentifier = bytes32(uint256(uint160(emittingContract)));
      _attestations[remoteChainId][bytes32(0)][senderIdentifier][payloadHash] = true;

      emit OutputProven(remoteChainId, bytes32(0), senderIdentifier, payloadHash);
  }
```

If some parameters become indexed in the actual event, adjust the topic extraction and data decoding accordingly.

LiFi: Fixed in PR 89.

Cantina Managed: Fix verified.

3.3.4 Bitcoin Oracle proof can be reused for multiple orders

Severity: Low Risk

Context: BitcoinOracle.sol#L364-L387

Summary: A Bitcoin transaction proof in the BitcoinOracle contract can be used to verify multiple, unrelated orders. This allows a malicious solver to fill one order legitimately and then reuse the same proof for other orders with identical payment parameters.

Description: The issue occurs because the `_verify()` function doesn't validate that a given Bitcoin transaction proof corresponds uniquely to a specific order. When verifying an output, the contract checks:

1. Whether the transaction has enough confirmations.
2. Whether the output script matches the expected Bitcoin script.
3. Whether the amount matches the expected amount.

However, it doesn't check that the transaction was specifically created for the claimed order. This means that any transaction with the correct recipient and amount can be used to verify any order with matching parameters.

DRAFT

The primary issue is that `orderId` is treated as an opaque identifier that isn't tightly coupled with the proof itself, allowing the same Bitcoin transaction to be used for multiple different orders.

Impact Explanation: The impact is medium to high. This vulnerability could allow a malicious solver to:

1. Legitimately fill one order.
2. Claim multiple other orders with identical payment parameters (same recipient and amount).
3. Provide the same proof for these other orders, effectively getting paid multiple times for a single Bitcoin transaction.

Additionally, if a user disputes the fraudulent claim, they would lose their collateral when the solver provides the valid (but reused) proof.

Likelihood Explanation: The likelihood is low as it requires specific conditions to exploit:

1. Multiple orders with identical payment parameters (same recipient and amount).
2. Orders must exist within the same verification window (1 day).
3. Order creators must not be aware of this vulnerability and implement their own protection measures.

Recommendation: Advise users to include unique identifiers in their transactions:

- Encourage using slightly different amounts (by a few satoshis) for each order.
- Document this limitation clearly for users to implement appropriate safeguards via off-chain solutions.

LiFi: Acknowledged.

Cantina Managed: Acknowledged.

3.3.5 Limited flexibility and precision in BTC-to-collateral price multiplier

Severity: Low Risk

Context: BitcoinOracle.sol#L375

Summary: The BitcoinOracle contract uses a fixed constant multiplier to determine collateral requirements for claiming orders, which may lead to inaccurate valuation when there are significant price fluctuations between Bitcoin and the collateral token.

Description: The BitcoinOracle contract uses a `collateralMultiplier` to determine how much collateral a solver must provide when claiming an order:

```
uint64 public immutable collateralMultiplier = 1;
```

This multiplier converts Bitcoin amounts to collateral token amounts when calculating required deposits:

```
uint256 collateralAmount = output.amount * multiplier;
```

The current implementation has several limitations:

1. The multiplier is fixed at deployment time and cannot be updated to reflect changing market conditions.
2. The value is stored as a `uint64`, which may lack sufficient precision for accurate BTC-to-collateral conversion.
3. There is no integration with external price oracles to dynamically adjust the conversion rate.

While the current approach provides a simple deterrent against fraudulent claims, it may become ineffective if the relative value of Bitcoin to the collateral token changes significantly.

The `OutputDescription` struct already includes a `fulfillmentContext` field specifically designed for "non-particular data that is used to encode non-generic behaviour for a filler," making it an ideal location to include a suggested collateral multiplier or price guidance.

Impact Explanation: The impact is low to medium. An inaccurate multiplier could lead to:

DRAFT

1. Insufficient collateral requirements if Bitcoin prices rise significantly, reducing the economic security of the system.
2. Excessive collateral requirements if Bitcoin prices fall, potentially blocking legitimate solvers from participating.
3. Suboptimal deterrent effect for the dispute mechanism, as the cost of creating "free options" would fluctuate with market conditions.

Likelihood Explanation: The likelihood is low to medium. Cryptocurrency prices are notoriously volatile, and significant price fluctuations between Bitcoin and the collateral token are common. The longer the contract operates without redeployment, the higher the chance that the fixed multiplier becomes significantly misaligned with market reality.

Recommendation: While the current implementation is designed to be simple and provide a basic deterrent, consider adding support for a custom `collateralMultiplier` parameter in the `fulfillmentContext` field of the `OutputDescription` with fixed-point precision:

```
// In BitcoinOracle.sol

// Fixed point precision for multiplier (6 decimals)
uint256 private constant PRECISION = 1e6;

/**
 * @notice Extracts the collateral multiplier from the fulfillmentContext if available
 * @dev Uses the fulfillmentContext of the output to determine pricing:
 * 0xB0 is a custom multiplier. Requires output.fulfillmentContext == 0xB0 | multiplier (8 bytes)
 * All other formats fall back to the contract's default multiplier
 */
function _getCollateralMultiplier(OutputDescription calldata output) internal view returns (uint64) {
    bytes calldata fulfillmentContext = output.fulfillmentContext;
    uint256 fulfillmentLength = fulfillmentContext.length;

    if (fulfillmentLength > 0) {
        bytes1 contextType = bytes1(fulfillmentContext[0]);

        // Check for custom multiplier format (0xB0 + 8 bytes for multiplier)
        if (contextType == 0xB0 && fulfillmentLength == 9) {
            uint64 customMultiplier;
            assembly ("memory-safe") {
                // Load the multiplier value (bytes 1-9)
                customMultiplier := calldataload(add(fulfillmentContext.offset, 0x01))
                // Shift right to get only the relevant 8 bytes
                customMultiplier := shr(192, customMultiplier)
            }
            return customMultiplier;
        }
    }

    // Fall back to the contract's default multiplier
    return collateralMultiplier;
}

// Update the claim function to use the custom multiplier if provided
function claim(bytes32 solver, bytes32 orderId, OutputDescription calldata output) external {
    if (solver == bytes32(0)) revert ZeroValue();
    if (orderId == bytes32(0)) revert ZeroValue();

    bytes32 outputId = _outputIdentifier(output);
    // Check that this order hasn't been claimed before.
    ClaimedOrder storage claimedOrder = _claimedOrder[orderId][outputId];
    if (claimedOrder.solver != bytes32(0)) revert AlreadyClaimed(claimedOrder.solver);

    uint64 multiplier = _getCollateralMultiplier(output);

    claimedOrder.solver = solver;
    claimedOrder.claimTimestamp = uint32(block.timestamp);
    claimedOrder.sponsor = msg.sender;
    claimedOrder.multiplier = multiplier;
    // The above lines acts as a local re-entry guard. External calls are now allowed.

    // Calculate collateral amount using fixed point precision
    uint256 collateralAmount = (output.amount * multiplier) / PRECISION;

    // Collect collateral from claimant
```

DRAFT

```
    SafeTransferLib.safeTransferFrom(COLLATERAL_TOKEN, msg.sender, address(this), collateralAmount);

    emit OutputClaimed(orderId, _outputIdentifier(output));
}
```

This approach follows the existing pattern in `CoinFiller` where specific byte identifiers (`0x00`, `0x01`) are used to signal different fulfillment behaviors. By introducing a new type (`0xB0`) for custom multipliers, the system can support user-defined pricing while maintaining compatibility with the existing contract structure.

LiFi: Fixed in commit PR 86.

Cantina Managed: Fix verified.

3.3.6 BtcPrism security considerations with confirmation requirements

Severity: Low Risk

Context: BitcoinOracle.sol#L82

Summary: The BtcPrism light client implementation has different security properties than full Bitcoin nodes, creating potential risks when using it for transaction verification in the BitcoinOracle contract. The recommended number of confirmations (NC) should account for both these differences and the potential value at risk.

Description: The BtcPrism light client, used by the BitcoinOracle contract to verify Bitcoin transactions, has different security guarantees compared to a full Bitcoin node:

1. It does not fully validate difficulty adjustments but instead uses a simplified "1/4 law" verification.
2. This means each block provides approximately 1/4 of the security guarantees of a fully verified Bitcoin block.
3. A minimum of 5 confirmations is needed to achieve "full Bitcoin security" against chain reorganizations.

DRAFT

The BitcoinOracle contract allows intent creators to specify the required number of confirmations in the token parameter, but lacks clear guidance on selecting appropriate values based on security requirements.

The primary concerns are:

1. Non-canonical chains: An attacker could potentially create a valid but non-canonical chain with transactions that would pass BtcPrism verification but would eventually be reorganized out of the canonical chain.
2. Delayed updates: Since BtcPrism is primarily updated by Catalyst (approximately every 5 blocks), there may be windows where the light client's view of the chain differs from reality.
3. Insufficient confirmation requirements: Users may underestimate the security requirements for higher-value transactions.

Recommendation: Consider adding documentation and comments in the code clearly explaining the security tradeoffs of different confirmation requirements and modify the `_getNumConfirmations()` function to enforce minimum confirmation requirements based loosely on transaction value:

```
/**
* @notice Returns the number of confirmations from the token.
* @dev SECURITY CONSIDERATIONS:
* - BtcPrism provides ~1/4 of the security of full node validation per block
* - Recommended minimums:
*   - 2 confirmations: <$20K
*   - 3 confirmations: <$100K
*   - 4 confirmations: <$200K
*   - 5+ confirmations: >$200K
* - 5+ confirmations provide full Bitcoin security against reorganizations
**/
function _getNumConfirmations(
    bytes32 token
    uint256 amount
) internal pure returns (uint8 numConfirmations) {
    numConfirmations = uint8(token >> 8);
    // Enforce minimum confirmation requirements based on amount
    // 10,000,000 sats  $200K at $20K/BTC
    // 5,000,000 sats  $100K at $20K/BTC
    // 1,000,000 sats  $20K at $20K/BTC
    if (amount > 10000000 && numConfirmations < 5) {
        numConfirmations = 5;
    } else if (amount > 5000000 && numConfirmations < 4) {
        numConfirmations = 4;
    } else if (amount > 1000000 && numConfirmations < 3) {
        numConfirmations = 3;
    } else if (numConfirmations < 2) {
        numConfirmations = 2;
    }
}
```

An intent with only 1 confirmation should not be allowed.

Then, update all calls to this function to include the output amount:

```
- uint256 numConfirmations = _getNumConfirmations(token);
+ uint256 numConfirmations = _getNumConfirmations(token, output.amount);
```

DRAFT

LiFi: Acknowledged.

Cantina Managed: Acknowledged.

3.3.7 BTC timestamp manipulation can block valid order verification

Severity: Low Risk

Context: BitcoinOracle.sol#L467

Description: *(This issue was identified by the Catalyst team during the review)* The `BitcoinOracle` contract contains a vulnerability where malicious Bitcoin miners could prevent legitimate order fills from being verified. The issue is in the `_resolveClaimed()` function which checks `fillTimestamp >= claimTimestamp`. This check can be manipulated due to the flexible nature of Bitcoin block timestamps.

Bitcoin protocol allows block timestamps to be within a specific range:

- Greater than the median time of the last 11 blocks.
- Less than the network adjusted time +2 hours.

This gives miners flexibility to set timestamps anywhere from approximately -1 to +2 hours from the actual time while still producing valid blocks. A malicious miner could deliberately set a block timestamp to be slightly before the `claimTimestamp`, causing the condition `fillTimestamp >= claimTimestamp` to fail.

If a legitimate solver claims an order and immediately fills it, the block containing their payment transaction could have its timestamp set to just before the claim timestamp. This would make it impossible to verify the order fulfillment through the standard verification process, even though the payment was actually made after the claim. Using the previous block's timestamp also wouldn't help in this scenario.

Recommendation: Remove the potentially problematic requirement.

```
  function _resolveClaimed(uint32 fillTimestamp, bytes32 orderId, OutputDescription calldata output) internal
  ↪ returns (bytes32 solver) {
      // ...existing code...

-     if (sponsor != address(0) && fillTimestamp >= claimTimestamp) {
+     if (sponsor != address(0)) {
          bool disputed = disputer != address(0);
          // ... rest of the function ...
      }
  }
```

The validity checks already constrain the claim to be verified within a day.

LiFi: Fixed in PR 91.

Cantina Managed: Fix verified.

3.3.8 `_getProofPeriod()` returns incorrect period

Severity: Low Risk

Context: BitcoinOracle.sol#L131-L141

Description: `_getProofPeriod()` is used to get the period the disputer has to wait to finalize the dispute. `_getProofPeriod()` will return the period based on the number of confirmations. The problem here is that due to the code error, when `numConfirmations` is 3, `_getProofPeriod()` returns `SEVEN_CONFIRMATIONS`. When `numConfirmations` is greater than 7, the returned period is 7 * 15 minutes longer than expected, forcing the disputer to wait longer.

```
function _getProofPeriod(
    uint256 confirmations
) internal pure returns (uint256) {
    unchecked {
        uint256 gammaDistribution = confirmations < 3
            ? (confirmations == 1 ? ONE_CONFIRMATION : (confirmations == 2 ? TWO_CONFIRMATIONS :
            ↪  THREE_CONFIRMATIONS))
            : (
                confirmations < 8
                    ? (confirmations == 4 ? FOUR_CONFIRMATIONS : (confirmations == 5 ? FIVE_CONFIRMATIONS :
                    ↪  (confirmations == 6 ? SIX_CONFIRMATIONS : SEVEN_CONFIRMATIONS)))
                    : 181 minutes + confirmations * TIME_PER_ADDITIONAL_CONFIRMATION
            );
        return gammaDistribution + LEAD_TIME;
    }
}
```

Recommendation:

```
  function _getProofPeriod(
      uint256 confirmations
  ) internal pure returns (uint256) {
      unchecked {
-         uint256 gammaDistribution = confirmations < 3
+         uint256 gammaDistribution = confirmations < 4
              ? (confirmations == 1 ? ONE_CONFIRMATION : (confirmations == 2 ? TWO_CONFIRMATIONS :
              ↪  THREE_CONFIRMATIONS))
              : (
                  confirmations < 8
                      ? (confirmations == 4 ? FOUR_CONFIRMATIONS : (confirmations == 5 ? FIVE_CONFIRMATIONS :
                      ↪  (confirmations == 6 ? SIX_CONFIRMATIONS : SEVEN_CONFIRMATIONS)))
-                     : 181 minutes + confirmations * TIME_PER_ADDITIONAL_CONFIRMATION
+                     : 181 minutes + (confirmations - 7) * TIME_PER_ADDITIONAL_CONFIRMATION
              );
          return gammaDistribution + LEAD_TIME;
      }
  }
```

LiFi: Fixed in PR 85.

Cantina Managed: Fix verified.

DRAFT

3.3.9 Zero-division error causes `CoinFillerWithFee` DoS

Severity: Low Risk

Context: CanCollectGovernanceFee.sol#L104-L111

Description: In `CoinFillerWithFee`, the `governanceFee` is initially 0 and the `governanceFee` can only be set through `setGovernanceFee()` → `applyGovernanceFee()`, which takes at least 7 days.

```
uint64 public governanceFee = 0;
...
function setGovernanceFee(
    uint64 _nextGovernanceFee
) external onlyOwner {
    if (_nextGovernanceFee > MAX_GOVERNANCE_FEE) revert GovernanceFeeTooHigh();
    nextGovernanceFee = _nextGovernanceFee;
    nextGovernanceFeeTime = uint64(block.timestamp) + GOVERNANCE_FEE_CHANGE_DELAY;

    emit NextGovernanceFee(nextGovernanceFee, nextGovernanceFeeTime);
}
```

When `governanceFee` is 0, `_calcFee()` will revert due to zero-division error, which will DOS CoinFillerWithFee for at least 7 days.

```
function _calcFee(uint256 amount, uint256 fee) internal pure returns (uint256 amountFee) {
    unchecked {
        // Check if amount * fee overflows. If it does, don't take the fee.
        if (amount >= type(uint256).max / fee) return amountFee = 0;
        // The above check ensures that amount * fee < type(uint256).max.
        // amount >= amount * fee / GOVERNANCE_FEE_DENOM since fee < GOVERNANCE_FEE_DENOM
        return amountFee = amount * fee / GOVERNANCE_FEE_DENOM;
    }
}
```

Recommendation:

```
  function _calcFee(uint256 amount, uint256 fee) internal pure returns (uint256 amountFee) {
+     if(fee == 0 ) return amountFee = 0;
      unchecked {
          // Check if amount * fee overflows. If it does, don't take the fee.
          if (amount >= type(uint256).max / fee) return amountFee = 0;
          // The above check ensures that amount * fee < type(uint256).max.
          // amount >= amount * fee / GOVERNANCE_FEE_DENOM since fee < GOVERNANCE_FEE_DENOM
          return amountFee = amount * fee / GOVERNANCE_FEE_DENOM;
      }
  }
```

LiFi: Fixed in PR 82.

Cantina Managed: Fix verified.

3.3.10 Duplicate outputs are only filled once in orders

Severity: Low Risk

Context: BaseFiller.sol#L54-L67

Description: The `BaseFiller` contract's `_fill()` function cannot process duplicate outputs in orders, which limits certain use cases. For instance, if an order creator wants to perform the same action multiple times (such as purchasing two identical NFTs or calling the same function on a remote contract twice), the current implementation only processes the first instance. The issue stems from how the contract tracks filled outputs using a mapping with the output hash as a key:

```
mapping(bytes32 orderId => mapping(bytes32 outputHash => FilledOutput)) _filledOutputs;
```

When processing an output, the function checks if it has already been filled:

```
bytes32 outputHash = OutputEncodingLib.getOutputDescriptionHash(output);
bytes32 existingSolver = _filledOutputs[orderId][outputHash].solver;

// Early return if we have already seen proof.
if (existingSolver != bytes32(0)) return existingSolver;
```

DRAFT

This means that identical outputs (same recipient, token, amount, and other parameters) generate the same `outputHash`, causing only the first instance to be processed. This limitation is acknowledged in the code comments:

```
/**
 * @dev The implementation strategy (verify then fill) means that an order with repeat outputs
 * (say 1 Ether to Alice & 1 Ether to Alice) can be filled by sending 1 Ether to Alice ONCE.
 * !Don't make orders with repeat outputs. This is true for any oracles.!
 */
```

Recommendation: Consider adding a nonce field to the `OutputDescription` structure to differentiate otherwise identical outputs. This would ensure that each output generates a unique hash, allowing order creators to include multiple identical actions in a single order, such as purchasing multiple identical NFTs or calling the same contract function multiple times. Alternatively, allow `OutputDescription.fulfillmentContext` to be used to include additional data, such as a nonce to differentiate outputs.

LiFi: Acknowledged.

Cantina Managed: Acknowledged.

3.4 Gas Optimization

3.4.1 Code optimization suggestions

Severity: Gas Optimization

Context: BaseFiller.sol#L34, PolymerOracle.sol#L93, PolymerOracle.sol#L95

Description and Recommendations:

1. Remove `ADDRESS_THIS` immutable and use address(this) directly.

   Replace the hard-coded immutable `ADDRESS_THIS` with a direct call to `address(this)`, saving gas and allowing deployment behind a proxy:

   ```
   - bytes32 immutable ADDRESS_THIS = bytes32(uint256(uint160(address(this))));

     function _IAmRemoteFiller(bytes32 remoteFiller) internal view virtual {
   -     if (ADDRESS_THIS != remoteFiller) revert WrongRemoteFiller(ADDRESS_THIS, remoteFiller);
   +     if (bytes32(uint256(uint160(address(this)))) != remoteFiller) revert
   ↪  WrongRemoteFiller(bytes32(uint256(uint160(address(this)))), remoteFiller);
     }
   ```

2. Optimize attestation storage structure.

   Replace the nested mapping structure with a single mapping using a hash of all parameters to reduce storage and lookup costs:

   ```
   - mapping(uint256 remoteChainId => mapping(bytes32 senderIdentifier => mapping(bytes32 application =>
   ↪  mapping(bytes32 dataHash => bool)))) internal _attestations;
   + mapping(bytes32 attestationHash => bool) internal _attestations;
   ```

   Then update the `_isProven` function:

   ```
     function _isProven(uint256 remoteChainId, bytes32 remoteOracle, bytes32 application, bytes32 dataHash)
     ↪  internal view returns (bool) {
   -     return _attestations[remoteChainId][remoteOracle][application][dataHash];
   +     bytes32 attestationHash = keccak256(abi.encodePacked(remoteChainId, remoteOracle, application,
   ↪  dataHash));
   +     return _attestations[attestationHash];
     }
   ```

3. Optimize proof iteration loop.

   Use direct offset calculations instead of incrementing within the loop, reducing the number of call-data operations:

DRAFT

```
- for (uint256 offset; offset < proofBytes;) {
+ uint256 start = proofSeries.offset;
+ uint256 end = start + proofBytes;
+ for (uint256 offset = start; offset < end;) {
    // Load the proof description.
    uint256 remoteChainId;
    bytes32 remoteOracle;
    bytes32 application;
    bytes32 dataHash;
    // Load variables from calldata to save gas compared to slices.
    assembly ("memory-safe") {
-       remoteChainId := calldataload(add(proofSeries.offset, offset))
+       remoteChainId := calldataload(offset)
        offset := add(offset, 0x20)
-       remoteOracle := calldataload(add(proofSeries.offset, offset))
+       remoteOracle := calldataload(offset)
        offset := add(offset, 0x20)
-       application := calldataload(add(proofSeries.offset, offset))
+       application := calldataload(offset)
        offset := add(offset, 0x20)
-       dataHash := calldataload(add(proofSeries.offset, offset))
+       dataHash := calldataload(offset)
        offset := add(offset, 0x20)
    }
```

4. Reuse computed value instead of recalculating.

   Use the already calculated `outputId` value instead of recomputing `_outputIdentifier(output)` in the emit statement:

```
   bytes32 outputId = _outputIdentifier(output);
   // Check that this order hasn't been claimed before.
   ClaimedOrder storage claimedOrder = _claimedOrder[orderId][outputId];
   if (claimedOrder.solver != bytes32(0)) revert AlreadyClaimed(claimedOrder.solver);
   uint64 multiplier = collateralMultiplier;

   claimedOrder.solver = solver;
   claimedOrder.claimTimestamp = uint32(block.timestamp);
   claimedOrder.sponsor = msg.sender;
   claimedOrder.multiplier = uint64(multiplier);
   // The above lines acts as a local re-entry guard. External calls are now allowed.

   // Collect collateral from claimant.
   uint256 collateralAmount = output.amount * multiplier;
   SafeTransferLib.safeTransferFrom(COLLATERAL_TOKEN, msg.sender, address(this), collateralAmount);

- emit OutputClaimed(orderId, _outputIdentifier(output));
+ emit OutputClaimed(orderId, outputId);
```

LiFi: Fixed in PR 88.

Cantina Managed: Fix verified.

3.5 Informational

3.5.1 OP Stack interoperability risks for fills and resource unlocks

Severity: Informational

Context: BaseFiller.sol#L82-L84

Description: OP Stack is developing new interoperability features with an unusual message validation pattern in their planned CrossL2Inbox contract. The key issue is that calls to validate invalid cross-chain messages will always be filtered out by the sequencer and never appear on-chain, resulting in what appears to be a "silent reorg" of transactions containing these calls.

This creates a potential attack vector through arbitrary calldata manipulation that could affect Catalyst protocol in indirect ways:

1. An intent creator could include a call to `crossL2Inbox.validateMessage(invalidMessage)` in their fulfillment logic.

DRAFT

2. When a solver attempts to fill an order with multiple outputs:
    - Initial simulation for all outputs would succeed.
    - First output might be filled successfully.
    - Second output's fulfillment would include the call to validate an invalid message.
    - This transaction would be silently dropped by the sequencer.
    - The solver would be unable to complete the fill despite spending resources on the first output.
    - Eventually, the intent would expire, causing the solver to lose their filled tokens.
3. Similarly, resource unlock functions could be targeted if they include external calls (e.g., token transfers) that might be manipulated to call OP Stack's message validation with invalid parameters, effectively blocking resource unlocks.

This behavior is particularly concerning because it breaks the standard assumption that invalid operations will explicitly revert on-chain, making it difficult to detect or prevent this form of attack through conventional means.

Recommendation:

1. Monitor OP Stack's implementation decisions closely as they continue developing their cross-chain messaging system and any potential mitigations for this issue.
2. If this feature is implemented as currently designed, consider requiring additional off-chain checks performed by the allocator that could:
    - Filter out non-whitelisted tokens that might contain malicious logic.
    - Disallow or carefully validate arbitrary calls within intent fulfillment paths.

3.5.2 Add `remoteChainId` non-zero check in `PolymerOracle._processMessage()`

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: In `PolymerOracle._processMessage()`, when `remoteChainId` is 0, it means that `_chainIdentifierToBlockChainId` is not initialized, and it should be reverted.

```
function _processMessage(
    bytes calldata proof
) internal {
    (uint32 chainId, address emittingContract, bytes memory topics, bytes memory unindexedData) =
    ↪  CROSS_L2_PROVER.validateEvent(proof);

    // Store payload attestations;
    bytes32 orderId = bytes32(topics[0]);

    (bytes32 solver, uint32 timestamp, OutputDescription memory output) = abi.decode(unindexedData, (bytes32,
    ↪  uint32, OutputDescription));

    bytes32 payloadHash = _proofPayloadHash(orderId, solver, timestamp, output);

    // Convert the Polymer ChainID into the canonical chainId.
    uint256 remoteChainId = _chainIdentifierToBlockChainId[chainId];
    bytes32 senderIdentifier = bytes32(uint256(uint160(emittingContract)));
    _attestations[remoteChainId][bytes32(0)][senderIdentifier][payloadHash] = true;

    emit OutputProven(remoteChainId, bytes32(0), senderIdentifier, payloadHash);
}
```

Recommendation:

```
  uint256 remoteChainId = _chainIdentifierToBlockChainId[chainId];
+ if (remoteChainId == 0) revert ZeroValue();
```

LiFi: Fixed in PR 83.

Cantina Managed: Fix verified.

DRAFT

3.5.3 `BitcoinOracle` should limit the maximum number of confirmations

Severity: Informational

Context: BitcoinOracle.sol#L366

Description: When the solver verifies the output, the proof will become invalid after more than one day, which is 1440 minutes.

```
function _verify(bytes32 orderId, OutputDescription calldata output, uint256 blockNum, BtcTxProof calldata
↪  inclusionProof, uint256 txOutIx, uint256 timestamp) internal {
    // Validate that the transaction happened recently:
    if (timestamp + CAN_VALIDATE_OUTPUTS_FOR < block.timestamp) revert TooLate();
```

Considering that the one block is 10 minutes, this means 144 confirmations. The current maximum confirmations is 255, which means that if the confirmations in the output is large, such as 200, even if the solver already has the proof, the solver cannot verify it because the proof will become invalid after 144 confirmations.

```
function _getNumConfirmations(
    bytes32 token
) internal pure returns (uint8 numConfirmations) {
    assembly ("memory-safe") {
        // numConfirmations = token << 240 [nc, utxo, 0...]
        // numConfirmations = numConfirmations >> 248 [...0, nc]
        numConfirmations := shr(248, shl(240, token))

        // numConfirmations = numConfirmations == 0 ? 1 : numConfirmations
        numConfirmations := add(eq(numConfirmations, 0), numConfirmations)
    }
}
```

Recommendation: It is recommended to cap the max confirmations in `claim()`, so the solver will not be compromised by claiming invalid orders.

LiFi: Fixed in PR 83.

Cantina Managed: Fix verified.

DRAFT

3.5.4 Code quality improvements

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description and Recommendations:

1. Improve order owner logic flow in `BaseSettler`.

   Simplify the order owner determination logic to make it more readable and easier to understand:

   ```
   orderOwner = address(uint160(uint256(solver)));
   if (purchaser != address(0)) {
       // ...
       delete purchaseDetails.lastOrderTimestamp;
       delete purchaseDetails.purchaser;
       if (lastOrderTimestamp <= orderTimestamp) {
           orderOwner = purchaser;
       }
   }
   return orderOwner;
   ```

2. Fix function naming capitalization.

   ```
   Correct capitalization in function name:
   ```

   ```
   - function getBlockChainIdtoChainIdentifier(
   + function getBlockChainIdToChainIdentifier(
   ```

3. Fix typo in Bitcoin confirmations documentation.

   Correct the spelling mistake in documentation:

```
- * 7 confimrations: 181 minutes.
+ * 7 confirmations: 181 minutes.
```

4. Fix grammar in `CompactSettler` documentation.

   Correct the subject-verb agreement in the documentation:

```
- * They then needs to either register or sign a supported claim with the intent as the witness.
+ * They then need to either register or sign a supported claim with the intent as the witness.
```

5. Add index to `OrderFilled` event.

   Index the orderId parameter in the `OutputFilled` event to allow for more efficient off-chain filtering and tracking:

```
- event OutputFilled(bytes32 orderId, bytes32 solver, uint32 timestamp, OutputDescription output);
+ event OutputFilled(bytes32 indexed orderId, bytes32 solver, uint32 timestamp, OutputDescription
↪  output);
```

6. Consider fixed point for constants if more precision is needed.

   The current approach using integer constants (`DISPUTED_ORDER_FEE_FRACTION = 3` and `CHALLENGER_COLLATERAL_FACTOR = 2`) is appropriate if these exact values are intended. However, if more precise values are needed in the future, consider using fixed-point representation.

7. Rename variable to clarify its purpose in `PolymerOracle`.

   Rename `senderIdentifier` to `application` in `PolymerOracle._processMessage()` to better reflect its purpose:

```
- bytes32 senderIdentifier = bytes32(uint256(uint160(emittingContract)));
- _attestations[remoteChainId][bytes32(0)][senderIdentifier][payloadHash] = true;
+ bytes32 application = bytes32(uint256(uint160(emittingContract)));
+ _attestations[remoteChainId][bytes32(0)][application][payloadHash] = true;
```

   Also update the related event emission:

```
- emit OutputProven(remoteChainId, bytes32(0), senderIdentifier, payloadHash);
+ emit OutputProven(remoteChainId, bytes32(0), application, payloadHash);
```

8. Overriding the `_domainNameAndVersion()` function in `CompactSettlerWithDeposit`.

   Override the `_domainNameAndVersion()` function in `CompactSettlerWithDeposit` to use the different name:

```
function _domainNameAndVersion() internal pure virtual override returns (string memory name, string
↪  memory version)     {
    name = "CatalystSettlerWithDeposit";
    version = "Compact1";
}
```

LiFi: Fixed in PR 87.

Cantina Managed: Fix verified.

DRAFT